

Qualitätsanalyse als Microservice-Prototyp

Vier Analyse-Services prüfen eine Engine-Konfiguration.
Status, Einzelresultate und Wiederholungen bleiben sichtbar.

REST-Choreographie mit Spring Boot, React und Docker Compose

Fallstudie und Anforderungen

Ausgangslage

Die Fallstudie bündelt Analysen in einer gekoppelten Komponente.
Lange Läufe und Ausfälle erschweren die Bedienung.

Ziel des Prototyps

Eigenständige Analyse-Services mit nachvollziehbarem Zustand und kontrollierter Wiederholung.

Konfiguration

Fünf Equipment-Felder persistent speichern

Ablauf

Vier Services in einer REST-Choreographie

Ergebnisse

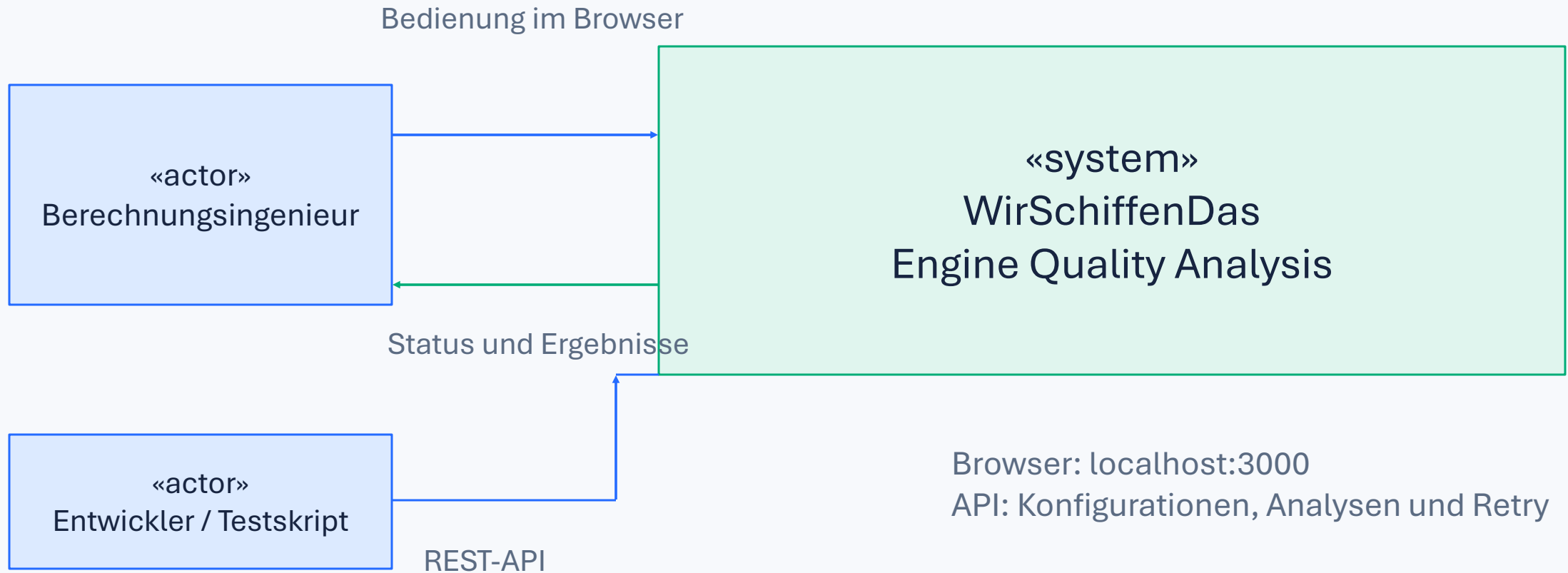
Einzelresultate je Equipment und ein Gesamtergebnis

Fehlerfall

Status anzeigen, Serviceausfall erkennen und Retry erlauben

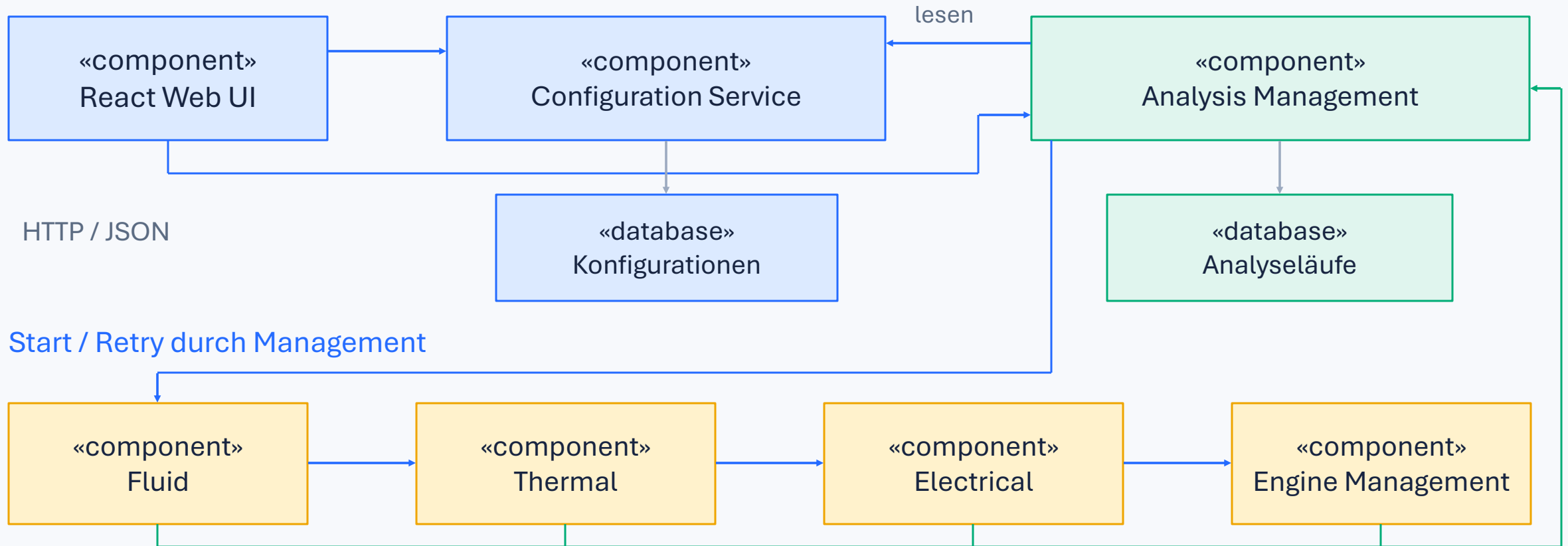
Kontextsicht

Systemgrenze, Nutzer und externe Zugänge



Bausteinsicht

Sechs Backend-Services, Weboberfläche und getrennte Datenhaltung

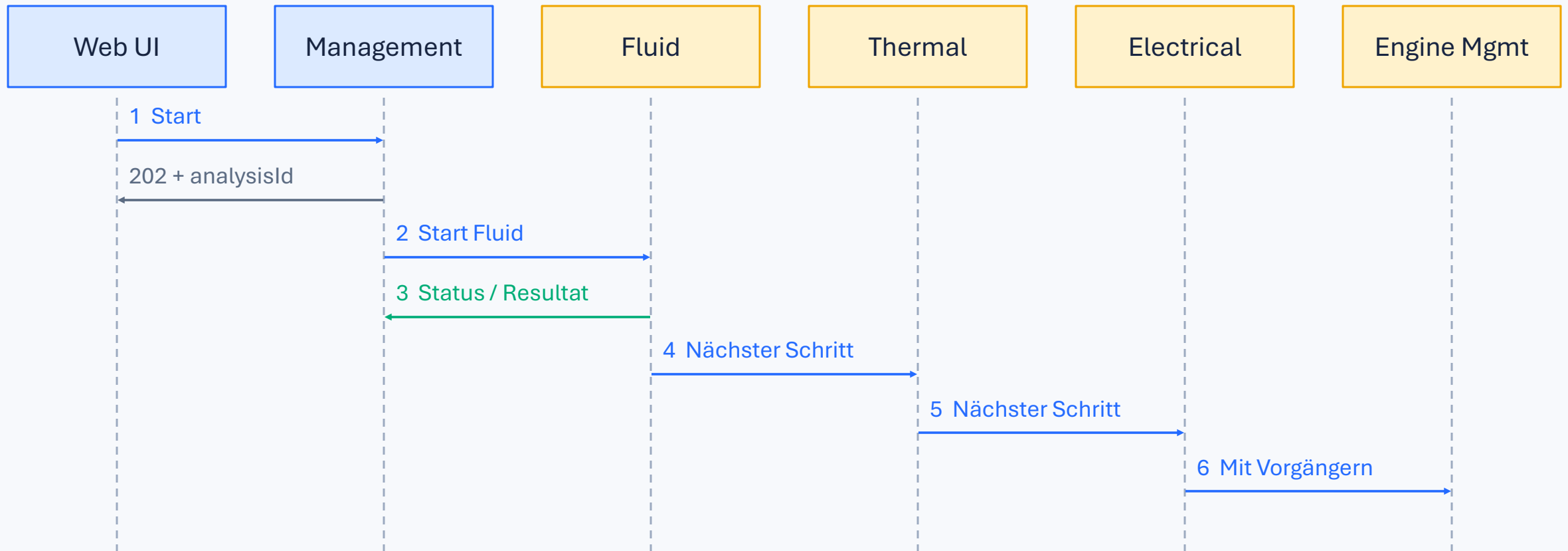


Status und Ergebnisse an Management

Jeder Worker kennt seinen Nachfolger. Nur zwei Services besitzen Datenbanken.

Laufzeitsicht

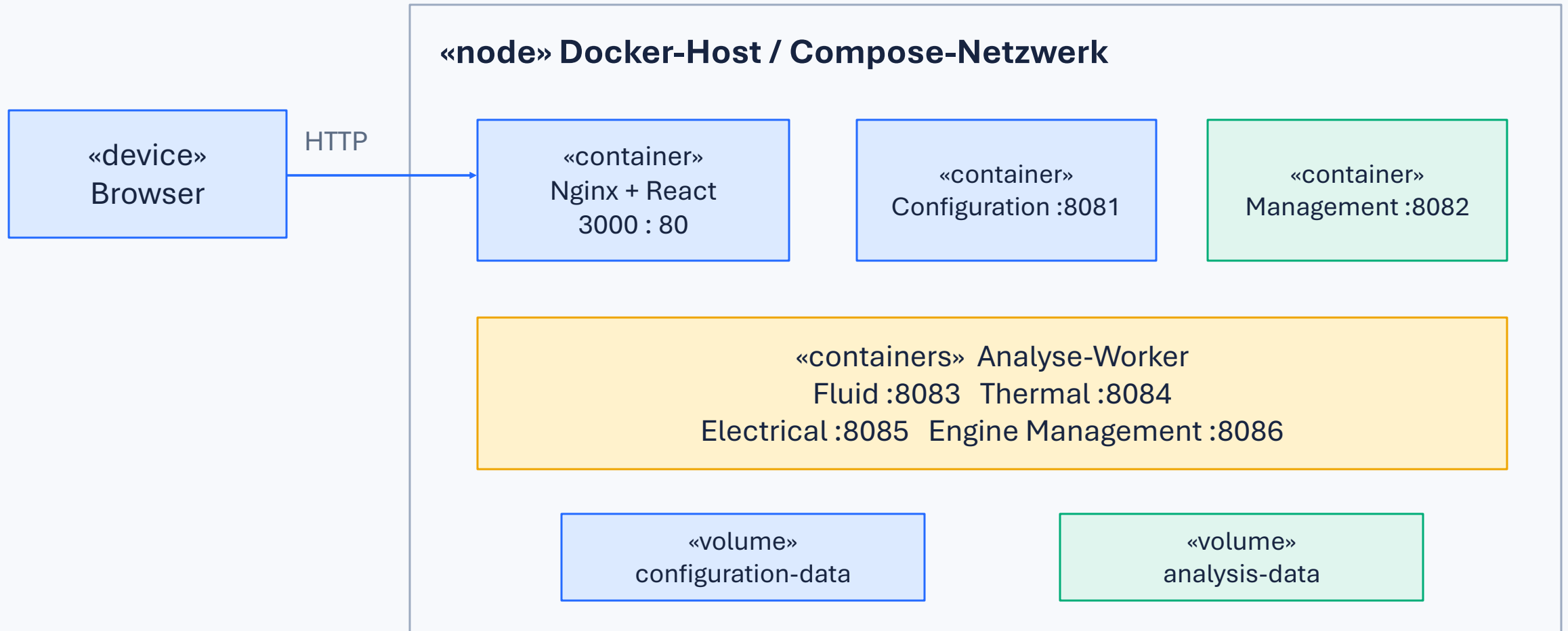
Asynchroner Gesamtauftrag mit synchronen HTTP-Übergaben



Jeder Worker meldet RUNNING und sein Ergebnis an Management.

Verteilungssicht

Ein Docker-Host für die lokale Demonstration



Build aus lokalen Quellen, feste npm-Abhängigkeiten und Readiness-Healthchecks

Entscheidung 1: Fachliche Services und Choreographie

Gewählt

Configuration und Analysis Management besitzen ihre Daten. Vier Worker verantworten ihre fachlichen Prüfungen.

Alternativen

Ein modularer Monolith spart Betriebskosten. Ein zentraler Orchestrator macht verzweigte Abläufe übersichtlicher.

Begründung

Die Zerlegung folgt den Analyseaufgaben. REST-Weitergabe erfüllt die Übungsanforderung und bleibt im Code sichtbar.

Preis der Entscheidung

Nachfolger und JSON-Verträge koppeln die Worker. DDD ist teilweise umgesetzt: Worker nutzen einfache DTOs und Maps.

Entscheidung 2: Sichere Zustände bei Übertragungsfehlern

Klare Ablehnung

Verbindung verweigert, Circuit Breaker offen oder HTTP 4xx: Zielschritt wird FAILED und kann wiederholt werden.

Absicherung

Breaker je Zielservice, begrenzte HTTP-Aufrufe und gespeicherter Inaktivitätszeitpunkt. Jeder Retry erhält eine neue attemptId.

Unklarer Ausgang

Timeout oder HTTP 5xx: Ausführung kann bereits laufen. Status bleibt offen, spätere Rückmeldung bleibt möglich.

Alternative und Grenze

Dauerhafte Nachrichten mit Outbox wären robuster. Der Prototyp nutzt Callback-Retries und einen Watchdog ohne Queue.

Code-Walkthrough

Drei Stellen erklären Fachlogik und Fehlerbehandlung

1 AnalysisWorker (Fluid)

Prüft Öl und Kraftstoff separat. Meldet vollständige Equipment-Ergebnisse und gibt bei Erfolg weiter.

2 AnalysisRun / AlgorithmExecution

Prüft vollständige Ergebnisse und zulässige Übergänge. Retry ersetzt Versuch-IDs ab dem gewählten Schritt.

3 NextServiceClient

Unterscheidet Ablehnung von unklarer Annahme. Der Circuit Breaker schützt genau diese Übergabe.

Live-Demo

Vier Fälle zeigen die fachlichen und technischen Grenzen

1

Gültige Konfiguration

Vier READY-Ergebnisse, fünf Equipment-Resultate OK, Gesamt-OK

2

Ungültiges Ölsystem

ÖL FAILED, Kraftstoff OK. Der Fluid-Service liefert beide Einzelresultate.

3

Thermal-Service gestoppt

Neue gültige Analyse starten. Fluid bleibt OK, Thermal schlägt fehl. Breaker der Weitergabe öffnen.

4

Wiederaanlauf und Retry

Thermal starten, Retry ausführen. Anschließend neuen Gesamtlauf als Probe der ursprünglichen Kante starten.

Bewertung mit Literatur und Tests

Shared Persistence

Separate Datenhoheit für Konfigurationen und Analyseläufe. Kein Service liest die Datenbank eines anderen.

Prüfbare Eigenschaften

Tests für Ergebnisvollständigkeit, Versuch-IDs, parallele Änderungen, Callback-Wiederholung sowie Ausfall und Wiederanlauf.

Timeouts und Failure Isolation

Begrenzte HTTP-Aufrufe, getrennte Breaker und Watchdog begrenzen Fehlerfolgen. Ein ausgefallener Zielservice blockiert andere Breaker nicht.

Offene Punkte

Lokale Logs und Healthchecks ersetzen kein verteiltes Tracing. API-Versionierung und automatische CI-Prüfungen fehlen noch.

Fazit und Grenzen

Die Analyse liefert vollständige Einzelresultate und einen nachvollziehbaren Weg durch Fehler und Retry.

Erkenntnis

Servicegrenzen allein sichern keinen Ablauf. Dafür braucht es Ergebnisverträge, Versuch-IDs und Regeln für unklare Zustellung.

Grenzen des Prototyps

Simulierte Fachlogik, lokale Worker-Deduplizierung und ein einzelner Docker-Host. Keine dauerhafte Queue und kein Hochverfügbarkeitsnachweis.

Nächste sinnvolle Schritte: dauerhafte Zustellung und bessere Betriebsbeobachtung